Voor jouw apotheek

SvelteKit Fundamentals

Server sided data loading

Peter Kassenaar – info@kassenaar.com

eduvision
opleiding & training

SvelteKit Fundamentals
Server sided data loading

Peter Kassenaar –
info@kassenaar.com

Why *server side data fetching*?

Mental model:

SvelteKit is not about

“fetch in components”

It is

“load data before rendering the page.”

That’s a different mental model from Angular/React

Traditionally – loading spinners

- Traditionally: components with lifecycle hooks
  - `onMount(() => fetch(…))`
  - Of course in a variety of syntax, depending on the framework
  - This is NOT WRONG, but a different approach
- In SvelteKit, we fetch data on/from the server
  - `load()` → pass data to page → page renders
  - E.g. The page already has data when it renders
  - No loading spinners necessary (a lot of times)

Data fetching from the server:

- Advantages
  - No CORS issues (!)
  - Safe to use (private or secret) API keys
  - Faster initial load
  - Better SEO by default (Google Ranking)
- Centralized data logic
  - Flow is much cleaner / nicer architecture.
  - `Route` → `load()` → `data` → components

Using a `load()` function

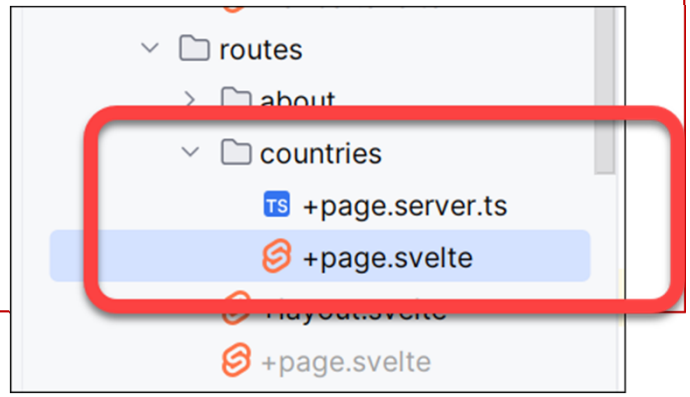
- The `load()` function lives in:
  - `+page.server.ts` → only runs on the server
  - `+page.ts` → runs on server and client
- The `load()` function returns an object
  - We can use TypeScript to <Type> the object
  - The object becomes available as `data` on the page
  - We use `PageServerLoad` type to indicate the function runs on the server

+page.server.ts – fetch data server-sided

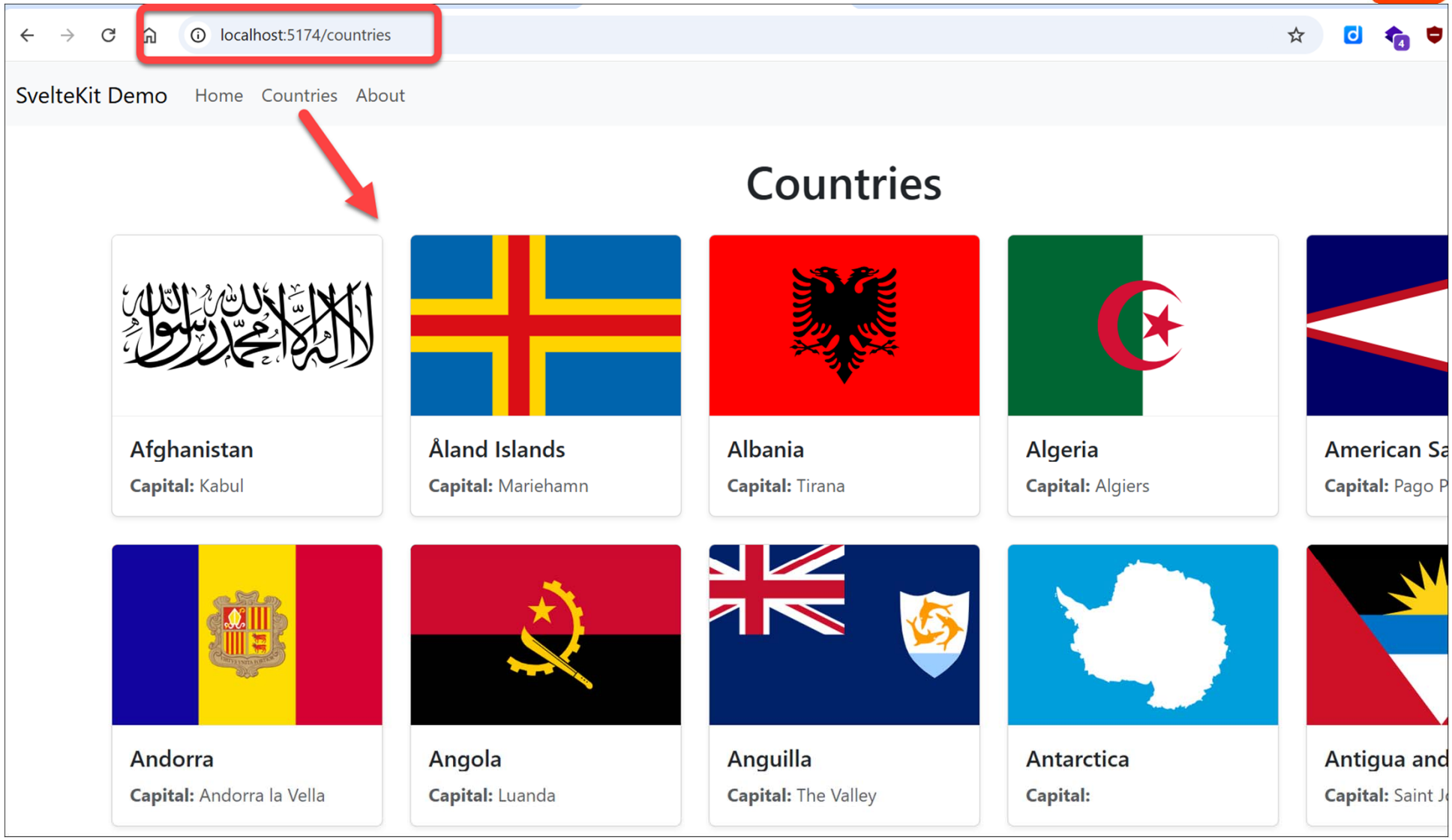
```
// countries/+page.server.ts
import type { PageServerLoad } from './$types';
import type { Country } from '../../types/Country';

export const load: PageServerLoad = async ({ fetch }) => {
  // 1. Fetch the countries from the API
  const url = 'https://restcountries.com/v3.1/all?fields=…';
  const res = await fetch(url);
  const countries: Country[] = await res.json();

  // 2. Sort the countries alphabetically (convenience, not required)
  countries.sort((a, b) => a.name.common.localeCompare(b.name.common));

  // 3. Pass to the page component
  return {
   countries
  };
};
```

+page.svelte – render data

```
<script lang="ts">
  import CountryCard from '$lib/CountryCard.svelte';
  import type { PageProps } from './$types';

  // 0. data is loaded from +page.server.ts
  let { data }: PageProps = $props();
</script>

<!--Simple UI. Just show the country-->
<h1 class="mb-4 text-center">Countries</h1>

<div class="…">
  <!-- Choice: only render the first 30 countries-->
  {#each data.countries.slice(0, 30) as country}
   <div class="col">
    <CountryCard {country} />
   </div>
  {/each}
</div>
```



Result



../450-sveltekit-server-load

More information

SVELTE Docs Tutorial Packages Playground Blog search

Getting started
- Introduction
- Creating a project
- Project types
- Project structure
- Web standards

Core concepts
- Routing
- Loading data
- Form actions
- Page options
- State management
- Remote functions

Build and deploy
- Building your app
- Adapters

Cookies

A server `load` function can get and set `cookies`.

src/routes/+layout.server.ts JS TS

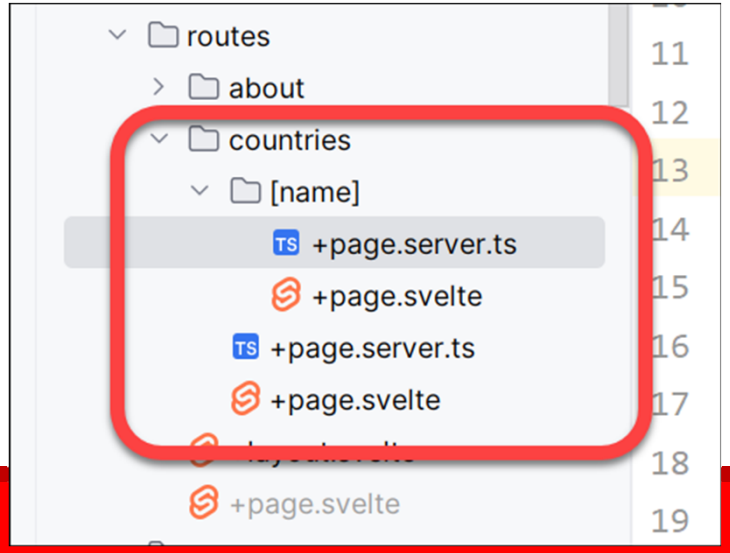
```
import * as db from '$lib/server/database';
import type { LayoutServerLoad } from './$types';

export const load: LayoutServerLoad = async ({ cookies }) => {
  const sessionid = cookies.get('sessionid');

  return {
    user: await db.getUser(sessionid)
  };
};
```

Cookies will only be passed through the provided `fetch` function if the target host is the same as the SvelteKit application or a more specific subdomain of it.

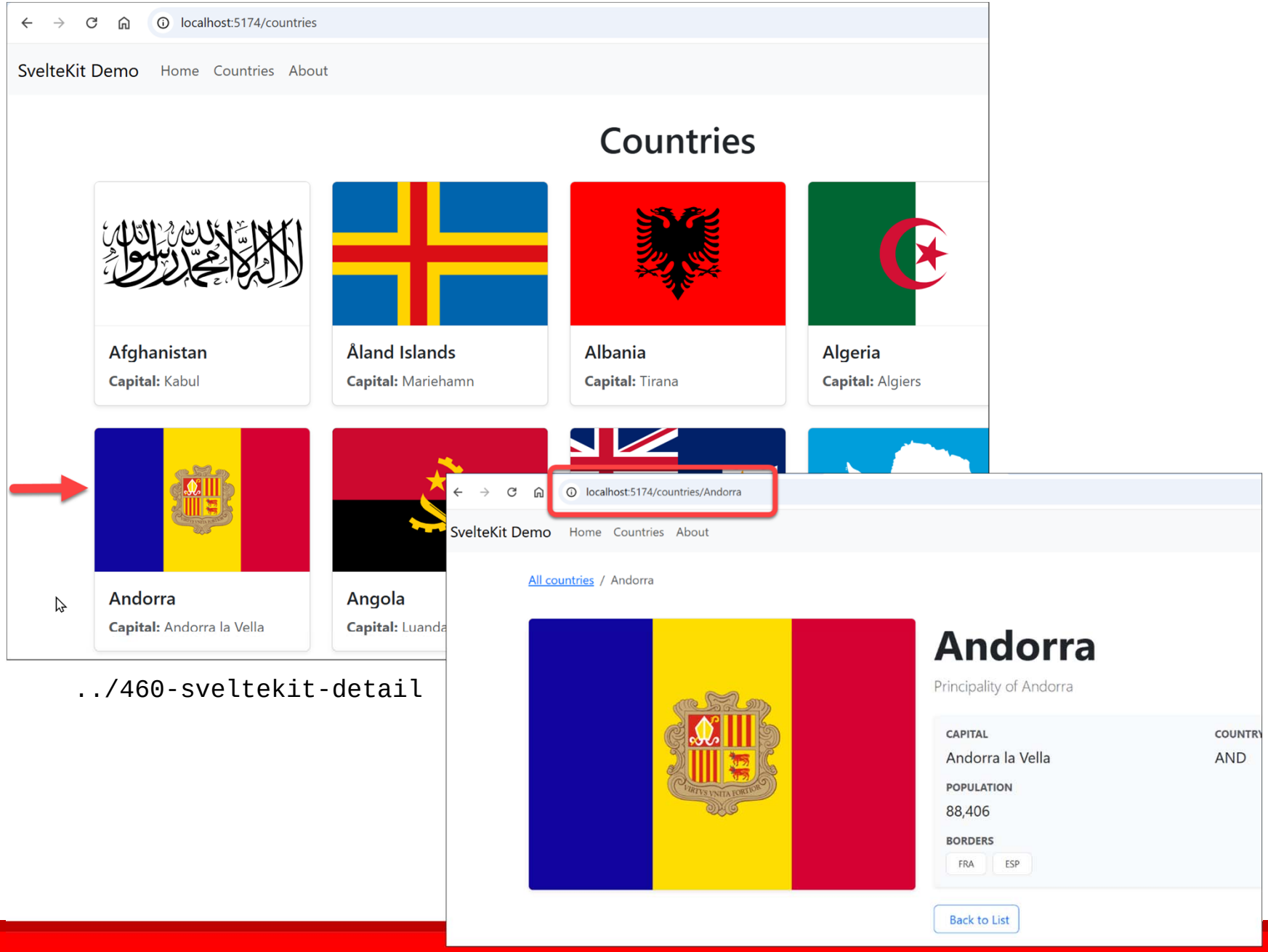
https://svelte.dev/docs/kit/load

Extra example – fetching details (server-sided)

- Make sure we can click a country.
  - Pass unique ID (here: `common name`. Often a `UUID` or `id`)
  - `<a href="/countries/{country.name.common}" >` in `CountryCard.svelte`
- Create a `/countries/[name]/page.server.ts` route
- Fetch country details inside that page, pass it on as `data`, as before.
  - Remember to update the URL
- Create a `+page.svelte` to render the passed country



Results



../460-sveltekit-detail

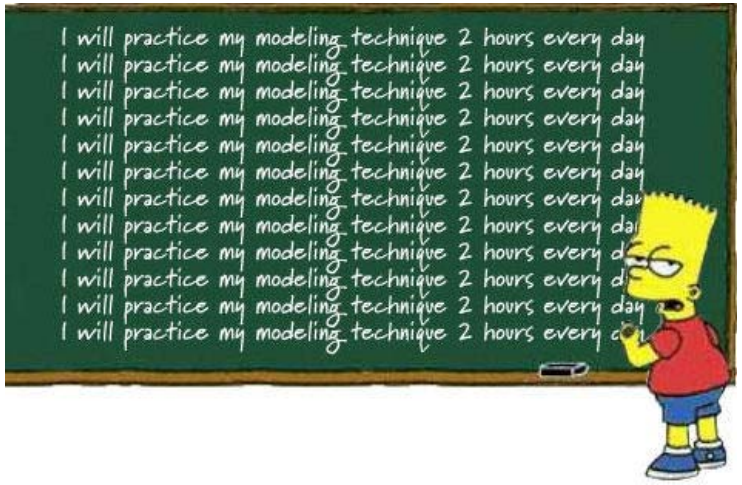
SvelteKit `load()` and `fetch()`

- Lesson: SvelteKit prefers `load()` over client-sided data fetching
- Server-sided `fetch()` is NOT browser `fetch()`!
  - It is SvelteKit-enhanced `fetch()`
  - Can handle cookies and HTTP headers automatically
  - Works on the server
  - Can call internal endpoints without full http roundtrip (=performance)

Summary

- Plain Svelte: `onMount()` + `fetch()`
- SvelteKit: `load()`

- In SvelteKit, data is loaded before rendering the page. Not afterwards.
  - This is an architectural design decision.

Workshop

- Load dummy users (or posts, or something else) from https://jsonplaceholder.typicode.com/users or use your own API
- Create a server-sided `fetch()` for the data
- Pass it on to a page component, display in a list
- Optional: enhance the list so you can click an item and fetch details
- Example: `…/460-sveltekit-server-detail`